



Tribhuvan University

Institute of Science and Technology

Supervisor's Recommendation

This is to certify that this project report prepared by Bibek Rawal, Prabesh Gautam and Rahul Koju on the topic “NLTTA: Nepali Language Translation and Transliteration Automation” in partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science and Information Technology (BSc. CSIT) is recommended for the final evaluation.

Mr. Sushant Paudel

Supervisor

BSc. CSIT Department

Bhaktapur Multiple Campus



Tribhuvan University

Institute of Science and Technology

Letter of Approval

This is to certify that this project report prepared by Bibek Rawal, Prabesh Gautam and Rahul Koju on the topic “NLTTA: Nepali Language Translation and Transliteration Automation” in partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science and Information Technology (BSc. CSIT) has been evaluated. In our opinion, it is satisfactory in the scope and quality of a project report for the required degree.

Mr. Sushant Paudel
Supervisor
BSc. CSIT Department
Bhaktapur Multiple Campus

Mr. Sushant Paudel
Coordinator
BSc. CSIT Department
Bhaktapur Multiple Campus

[Internal]
Internal Examiner
BSc. CSIT Department
Bhaktapur Multiple Campus

External Examiner

Acknowledgement

I would like to thank Tribhuvan University Institute of Science and Technology for giving me this opportunity via the course of Computer Science and Information Technology to help me understand the project ethics at this early stage and helped me to evaluate my knowledge and expand it a little more.

I would like to express my gratitude to Mr. Sushant Paudel, B.Sc. CSIT Coordinator at Bhaktapur Multiple Campus, for his assistance during my academic term and for creating an interactive environment at the institute that has assisted me in moving forward and choosing a career path.

My heartfelt thanks go to Supervisor, Mr. Sushant Paudel for providing continuous guidance, valuable insights, and unwavering support throughout the entire project. The project would not have been a success without his continuous supervision and encouragement.

I would like to extend my sincere gratitude to all those who contributed to the successful completion of my project. My project would not have been successful without the help and support of many individuals. I am blessed to have all those helping hands along me in the duration of my project.

Bibek Rawal (28294/078)

Prabesh Gautam (28294/078)

Rahul Koju (28294/078)

ABSTRACT

This project presents a complete pipeline for translating Romanized Nepali queries into English and retrieving the most relevant news articles using a fine-tuned mBART model called NLTTA and BM25 retrieval algorithm. We fine-tuned the multilingual mBART model on a parallel corpus of Romanized Nepali and English sentences to enhance its translation accuracy for Romanized (transliterated) languages, low-resource language input. The fine-tuned model is deployed via a Django-based REST API, which serves translation functionality in real time. Once a Romanized Nepali query is translated into English, the system applies the BM25 ranking algorithm on a pre-indexed English news database to retrieve and rank the most relevant articles based on the query. This hybrid system bridges the linguistic gap for Romanized Nepali speakers, enabling effective English news search and information access through a robust combination of neural translation and probabilistic information retrieval.

Keywords: BM25, mBART, Neural Translation, pre-indexed, Probabilistic retrieval, REST API

TABLE OF CONTENTS

Supervisor’s Recommendation	i
Letter of Approval	ii
Acknowledgement	iii
ABSTRACT.....	iv
TABLE OF CONTENTS	v
LIST OF TABLES	viii
LIST OF FIGURES	ix
LIST OF ABBREVIATIONS	x
CHAPTER 1: INTRODUCTION	1
1.1 Introduction.....	1
1.2 Problem Statement	1
1.3 Objectives	2
1.4 Scope and Limitation	2
1.5 Development Methodologies.....	3
Technology and Languages:.....	3
Frameworks:	3
Documentation:.....	4
1.6 Report Organization	4
Chapter 2: Background Study and Literature Review	5
2.1 Background Study.....	5
2.1.1 Deep Learning:.....	5
2.1.2 Transformers:	5
2.1.3 mBART:	5
2.1.4 Web Applications:	5
2.1.5 BM25:	6

2.2 Literature Review	6
2.2.1 Key Insights for the Project.....	7
Chapter 3: System Analysis	8
3.1 Requirement Analysis.....	8
3.1.1 Functional Requirements.....	8
3.1.2. Non-Functional Requirements	10
3.2 Feasibility Analysis	11
3.2.1. Technical Feasibility	11
3.2.2. Operational Feasibility	11
3.2.3. Economic Feasibility.....	11
3.2.4. Schedule Feasibility	11
3.3 Analysis	12
3.3.1 Class Diagram	12
.....	13
3.3.2 State Diagram.....	13
3.3.3 Sequence Diagram	14
Chapter 4: System Design.....	16
4.1 Design.....	16
4.1.1 Component Design.....	16
4.2 Algorithm Details	17
Chapter 5: Implementation and Testing.....	19
5.1. Implementation	19
5.1.1 Tools Used.....	19
5.1.2 Implementation Details of Modules	19
5.2 Testing.....	20
5.2.1 BLEU and BERT Score.....	20
5.3 Result Analysis	21

5.3.1 BLEU Score Result:.....	21
5.3.2 BERT Score Result:	22
Chapter 6: Conclusion and Future Recommendations.....	23
6.1 Conclusion	23
6.2 Future Recommendations	23
REFERENCES.....	24
APPENDIX.....	25

LIST OF TABLES

Table 1.5: Report Organization	4
Table 3.2.4: Schedule	11
Table 5.2.1: Test cases for Unit Testing	20

LIST OF FIGURES

Figure 3.1: Use Case Diagram for User and Admins.....	9
Figure 3.2.4: Gantt Chart.....	12
Figure 3.3.1: Class Diagram.....	13
Figure 3.3.2: State Diagram	14
Figure 3.3.3: Sequence Diagram.....	15
Figure 4.1: Component Diagram.....	16
Figure 5.3.1: : BLEU score vs Other Models	22
Figure 5.3.2: : BERT score vs Other Models	22

LIST OF ABBREVIATIONS

- ❖ **API** - Application Programming Interface
- ❖ **BART** - Bidirectional and Auto-Regressive Transformer
- ❖ **BERT** - Bidirectional Encoder Representations from Transformers
- ❖ **BLEU** - Bilingual Evaluation Understudy
- ❖ **BM25** - Best Matching 25 (Ranking Algorithm)
- ❖ **CNN** - Convolutional Neural Network
- ❖ **CPU** - Central Processing Units
- ❖ **DL** - Deep Learning
- ❖ **E-commerce** - Electronic Commerce
- ❖ **GPU** - Graphical Processing Units
- ❖ **IDF** - Inverse Document Frequency
- ❖ **JSX** - JavaScript XML
- ❖ **LLM** - Large Language Model
- ❖ **mBART** - Multilingual Bidirectional and Auto-Regressive Transformer
- ❖ **ML** - Machine Learning
- ❖ **MuRIL** - Multilingual Representations for Indian Languages
- ❖ **NLTTA** - Nepali Language Translation and Transliteration Automation
- ❖ **MYSQL** - My Structured Query Language
- ❖ **NMT** - Neural Machine Translation
- ❖ **REST** – Representational State Transfer

- ❖ **RNN** - Recurrent Neural Network
- ❖ **Seq2Seq** - Sequence to Sequence
- ❖ **TC** - Test Case
- ❖ **TF** - Term Frequency
- ❖ **TF-IDF** - Term Frequency–Inverse Document Frequency
- ❖ **TPU** - Tensor Processing Units
- ❖ **UI** - User Interface

CHAPTER 1: INTRODUCTION

1.1 Introduction

In Nepal, many people commonly communicate using a mixture of Nepali (both in Devanagari and Romanized scripts) and English, especially in informal digital contexts such as chatting, searching, and social media. Nepali is spoken by over 33 million people worldwide including both native and second-language speakers. However, most existing digital platforms and search engines are not designed to understand such code-mixed or informal queries. This creates a significant barrier for many users, especially those who are partially literate or not fluent in English. Thus, making accessing relevant and meaningful information online a hassle.

Neural machine translation (NMT) [1] is known to give state-of-the-art translations for a variety of language pairs. NMT is known to perform poorly for language pairs for which parallel corpora are scarce. This happens due to lack of translation knowledge as well as due to overfitting which is inevitable in a low-resource setting. Fortunately, transfer learning via cross-lingual transfer [2] can significantly improve translation quality in a low-resource situation.

To address this challenge, we introduce NLTTA (Nepali Language Translation and Transliteration Automation), a deep learning-based text understanding model specifically designed to process Nepali language in both Devanagari and Romanized forms, as well as mixed English-Nepali inputs. NLTTA can transliterate Devanagari to Romanized Nepali, translate text between Nepali and English, and understand informal, chat-style user inputs. The system aims to improve digital accessibility and enhance the usability of applications such as news search engines or product search on E-commerce sites for Nepali users. Through the integrating of our translation and transliteration system, our news platform delivers more accurate and inclusive search results, enhancing user experience and accessibility for a multilingual audience.

1.2 Problem Statement

In Nepal, a significant portion of the population is only partially literate or has limited fluency in English. As a result, many users rely on informal communication using Romanized Nepali

(e.g., “sansad” > “parliament”, “छात्रवृत्ति” > “scholarship”) or mix English with Nepali while typing. Unfortunately, current search engines and digital systems are not equipped to understand these types of informal or bilingual inputs.

This leads to several problems:

- ❖ Users receive irrelevant or no results when using Romanized or mixed-language queries.
- ❖ Nepali-speaking users face difficulty in accessing online information effectively.
- ❖ People with low English proficiency are digitally excluded.

Therefore, there is a strong need for a system that can:

- ❖ Understand both Devanagari and Romanized Nepali.
- ❖ Accurately transliterate and translate between Nepali and English.
- ❖ Handle informal inputs to return relevant results.

1.3 Objectives

This project aims to develop a model that understands Nepali in both Devanagari and Romanized scripts, handles informal mixed-language inputs, and improves access to digital content for users with limited English proficiency and integrate it with a web application for searching news contents.

The main objectives of this project are:

- ❖ To build a deep learning model that can understand and process text in Nepali (both Devanagari and Romanized forms).
- ❖ To understand informal, mixed, or chat-style Nepali-English language inputs.
- ❖ To integrate this system into a news search application that allows users to search using natural, informal Nepali.
- ❖ To improve digital inclusion for partially literate users and those not fluent in English.

1.4 Scope and Limitation

Our NLTTA application addresses a critical linguistic barrier in Nepal by enabling users to interact with digital systems using Devanagari Nepali, Romanized Nepali, and mixed-language (English-Nepali) inputs. It allows users especially those with limited English proficiency or partial literacy to search and access content more effectively, making it particularly impactful in rural communities. The system is designed to process informal, chat-style inputs and return

accurate translations and relevant news results. NLTTA can be integrated into various domains like news search platforms, e-commerce, and public service portals. In the developing digitization of Nepal, NLTTA ensures inclusive access to information and enhances digital participation for a broader population segment.

The limitations for our project are:

- ❖ The system's translation accuracy relies on the quality and diversity of the training corpus, which may need further enhancement to fully support complex informal expressions.
- ❖ Real-time performance may be limited in low-bandwidth areas, especially during model inference or API communication.
- ❖ Romanized Nepali lacks standardization, which can introduce variability and challenges in accurately interpreting user intent.
- ❖ The fine-tuned models, while effective, require ongoing updates and domain-specific tuning to maintain relevance and accuracy over time.

1.5 Development Methodologies

The application was developed using prototyping methodology. Due to the limitation on compute available we trained our model for few epochs tweaking hyperparameters to find out the best for our approach with each prototype.

Technology and Languages:

- Python
- TailwindCSS
- JavaScript

Frameworks:

- Django
- PyTorch
- Transformers
- Next.JS

Documentation:

- Microsoft word

1.6 Report Organization

Table 1.5: Report Organization

Chapter	Heading	Contents (Topic)
Chapter-1	Introduction	1.1 Introduction 1.2 Problem Statement 1.3 Objectives 1.4 Scope and Limitation 1.5 Development Methodology 1.6 Report Organization
Chapter-2	Background Study and Literature Review	2.1 Background Study 2.2 Literature Review
Chapter-3	Analysis	3.1 System Analysis 3.1.1 Requirement Analysis 3.1.2 Feasibility Analysis 3.1.3 Analysis
Chapter-4	System Design	4.1 Design 4.2 Algorithm Details
Chapter-5	Implementation and Testing	5.1 Implementation 5.2 Testing 5.3 Result Analysis
Chapter-6	Conclusion and Future Recommendations	6.1 Conclusion 6.2 Future Recommendations

Chapter 2: Background Study and Literature Review

2.1 Background Study

The development of our web application for retrieving by translation of romanized and mixed Nepali involves several fundamental theories, concepts, and terminologies. These include:

2.1.1 Deep Learning:

DL is a subset of machine learning that uses neural networks with multiple layers to model and understand complex patterns in data. From the many deep learning models, we are using transformer architectures for translating Nepali and handling mixed-language queries. These models learn contextual relationships across words and scripts, enabling more accurate translations and understanding of informal inputs.

2.1.2 Transformers:

Transformers are a deep learning architecture based on self-attention mechanisms. Unlike earlier sequential models like RNNs, transformers process entire sequences in parallel, capturing long-range dependencies and contextual relationships efficiently. It excels in sequence-to-sequence tasks like translation and text generation by capturing relationships between elements in a sequence.

2.1.3 mBART:

mBART is a sequence-to-sequence language model developed by Facebook AI for multilingual translation tasks. It is trained on multiple languages simultaneously and can handle translation and text generation across diverse language pairs. mBART is foundationally built upon BART model but using tags for support of different languages. It uses 12 layers of encoder and decoder of the Transformer architecture.

2.1.4 Web Applications:

Web-based platforms that enable users to perform specific tasks through an internet browser. They are highly accessible and scalable, making them ideal for deploying AI-powered tools to users in remote areas. We are designing and launching our application using Django framework developed in Python programming language for backend and NextJS for frontend.

2.1.5 BM25:

BM25 is a ranking algorithm used in information retrieval to determine the relevance of documents to a given search query. It improves upon earlier models like TF-IDF by considering term frequency saturation and document length normalization, making it highly effective for ranking search results. BM25 recognizes that the impact of term frequency plateaus after a certain point. Adding more occurrences of a term doesn't proportionally increase the document's relevance score.

2.2 Literature Review

Processing bilingual or code-mixed languages such as Nepali and English presents significant challenges, especially due to the low-resource nature of the Nepali language. Various works have attempted to address similar challenges in related languages, particularly in South Asian contexts, using transformer-based models and multilingual pre-training approaches.

One of the most relevant works is MuRIL (Multilingual Representations for Indian Languages) [2], was developed using a transformer-based language model tailored for Indian languages including Hindi, Tamil, and others. It used transliterated Roman script data along with native script data to improve understanding across diverse linguistic forms. This is directly relevant to our work on Nepali, where Romanized Nepali is common in informal contexts. MuRIL demonstrated that pre-training using both native and Romanized scripts significantly enhances downstream task performance, especially for sentiment analysis and question answering.

mBART [3] introduced a sequence-to-sequence pre-trained model for multilingual translation tasks. This model enabled better cross-lingual understanding and translation by using a denoising objective during pre-training. Its ability to generalize across language pairs without requiring large parallel datasets made it suitable for low-resource language applications like Nepali-English.

The SentencePiece tokenizer [4], is a language-agnostic subword tokenizer that enables better vocabulary representation in neural machine translation systems. This is especially important for morphologically rich and agglutinative languages such as Nepali, where word-level tokenization can be inefficient.

Transformers [5] serve as the base for all modern NLP models including BERT, mBART, and MuRIL. Its attention mechanism allows models to capture long-range dependencies, essential for effective translation and semantic understanding.

In terms of information retrieval, traditional methods like TF-IDF (Term Frequency–Inverse Document Frequency) and BM25 have proven effective in indexing and ranking documents. While neural methods like dense embeddings [6] are gaining traction, in this project, we choose to implement TF-IDF due to its simplicity, interpretability, and effectiveness in keyword-based document search. It enables fast and relevant retrieval of news articles based on bilingual (translated and transliterated) user queries.

NepaliBERT [7], is a recent advancement that trained a masked language model specifically on a Nepali corpus. While it focuses only on Devanagari script Nepali, it demonstrates the effectiveness of fine-tuning transformer models on Nepali text. However, it does not handle Romanized Nepali or bilingual input, which limits its application in real-world scenarios like search or chat-based interfaces.

2.2.1 Key Insights for the Project

- The reviewed literature confirms that transformer-based models like mBART are highly effective for multilingual tasks, particularly for low-resource languages when fine-tuned on domain-specific datasets.
- Transformers capture context well: Their attention mechanism helps translate mixed or informal bilingual inputs accurately.
- BM25 remains practical: Despite neural advances, BM25 offers fast, interpretable document retrieval suitable for real-time search.

This review supports the feasibility of the proposed project and provides valuable insights for overcoming potential challenges, ensuring the solution is practical and impactful.

Chapter 3: System Analysis

3.1 Requirement Analysis

3.1.1 Functional Requirements

Functional requirements describe the core functionalities of the web application. Below are the key functional requirements illustrated using a use case description.

Use Case Descriptions

a. Enter Query

- **Actor:** User
- **Description:** The user enters a search query in English, Devanagari Nepali, or Romanized Nepali.
- **Precondition:** The user is connected to the server and has access to the search interface.
- **Postcondition:** The query is passed to the system for preprocessing.

b. Translate Text

- **Actor:** System
- **Description:** The system translates the input text to a common language (usually English) if required for search relevance.
- **Precondition:** The user entered the query.
- **Postcondition:** A translated version of the query is generated.

c. Search News

- **Actor:** System
- **Description:** The system performs a cross-lingual search using the processed query to retrieve relevant news articles.
- **Precondition:** A translated/transliterated query is available.

- **Postcondition:** A ranked list of matching news articles is generated.

d. **View Result**

- **Actor:** User
- **Description:** The user views a ranked list of news articles that match their query, including title, image, and summary.
- **Precondition:** A search has been completed and results are available.
- **Postcondition:** The results are rendered on the interface.

e. **Manage System**

- **Actor:** Admin
- **Description:** The admin manages system configuration, settings, and backend operations.
- **Precondition:** Admin is authenticated.
- **Postcondition:** System updated with new news in database or model changes.

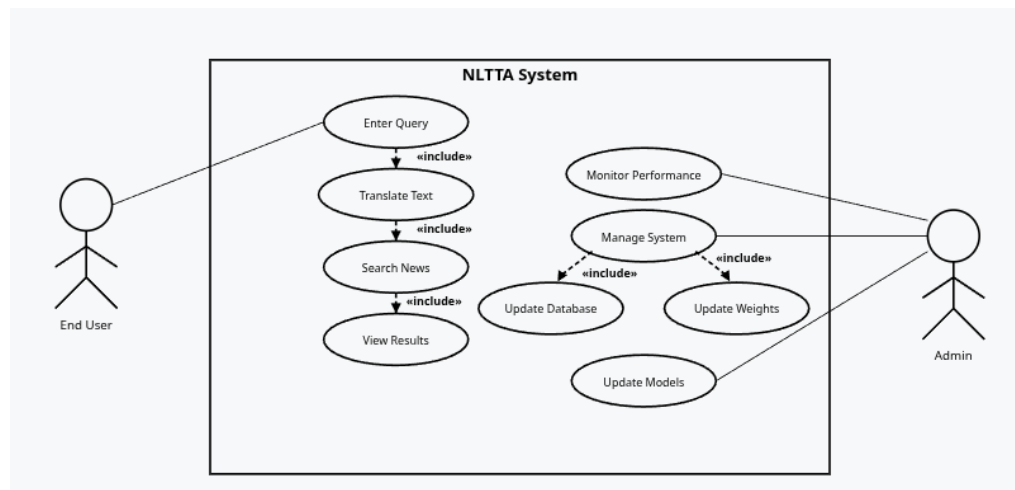


Figure 3.1: Use Case Diagram for User and Admins

3.1.2. Non-Functional Requirements

Non-functional requirements describe the system's overall qualities and constraints.

a. **Performance**

- The system should process any query and provide relevant news within 3 seconds.
- The relevant news retrieved must have accuracy rate above 80%.

b. **Scalability**

- The system should handle concurrent users and support the addition of news updates along with model fine tuning for future.

c. **Accessibility**

- The web application should be optimized for mobile devices and available in multiple languages, including Nepali.

d. **Usability**

- The interface should be user-friendly, requiring minimal technical knowledge for operation.

e. **Maintainability**

- The system should allow easy updates to the transformer model and software components on both frontend and backend.

f. **Connectivity**

- The application should function effectively in areas with limited bandwidth by optimizing image uploads and system response times.

These requirements form the foundation for designing and implementing a robust, user-centric disease detection system tailored for Nepalese farmers.

3.2 Feasibility Analysis

3.2.1. Technical Feasibility

We plan to utilize a transformer-based deep learning models such as mBART as the base, which are known for handling cross-lingual tasks. These models will be fine-tuned on a Nepali-English parallel corpus (extracted from Wikipedia dumps and semi-manually annotated Romanized Nepali data). The biggest technical hurdle would be the resource to train such a model which is solved by utilization of checkpoints saving. The checkpoint feature will allow us to utilize Google Colab provided resources to train and test the model for parts of data everyday over the period of the project.

3.2.2. Operational Feasibility

Our translation model will be deployed via an API in Python which we will integrate into our news searching platform. The API will be developed in Python using Flask framework, with the model running on GPU-supported environments or CPU if the GPU memory is limited. The news platform backend will be done using Django for backend and NextJs for the frontend.

3.2.3. Economic Feasibility

We aim to use freely and publicly available platform like Google Colab for training compute and publicly available datasets, annotated semi-manually to minimize cost. We will also be utilizing the pre-trained models thus making the cost of the project minimal. Any future deployment could be scaled using cloud platforms like Azure.

3.2.4. Schedule Feasibility

The project is achievable in 4 months:

Table 3.2.4: Schedule

S.No.	Task Name	Duration	Preceding Task
1	Literature Review and Data Collection	1 week	0
2	Preprocessing	1 week	1
3	Model Selection	1 week	2
4	Training and Fine Tuning	9 weeks	3

5	Web App Development	7 weeks	3
6	Document Indexing	3 weeks	5
7	API Development and Integration	2 weeks	4,6
8	Testing and Re-adjustment	1 week	7
9	Final Report and Documentation	1 week	8

Gantt Chart:

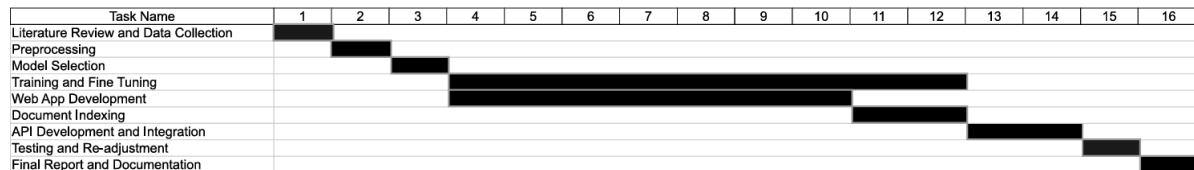


Figure 3.2.4: Gantt Chart

3.3 Analysis

The application is developed using object-oriented approach.

3.3.1 Class Diagram

Classes:

- ❖ **User:** Handles user queries and triggers translation processes.
- ❖ **Query:** Represents a user's search text and its translation, and performs translation operations.
- ❖ **Translator:** Manages text normalization, tokenization, and translation of incoming text into English.
- ❖ **Feed:** Represents content feeds in different languages, including their metadata and management operations.
- ❖ **FeedItem:** Represents individual items within a feed, such as articles or posts, with details like title and content.
- ❖ **BM25:** Implements the BM25 ranking algorithm for information retrieval, including text preprocessing and scoring functions.

Relationships:

- ❖ User enters a query, which creates a Query object.

- ❖ Each Query is processed by one Translator for normalization and translation.
- ❖ BM25 interacts with Translator to preprocess and tokenize texts for ranking.
- ❖ BM25 ranks documents in the Feed based on the user Query.
- ❖ A single Feed contains multiple FeedItem objects, managing the feed's items and metadata.

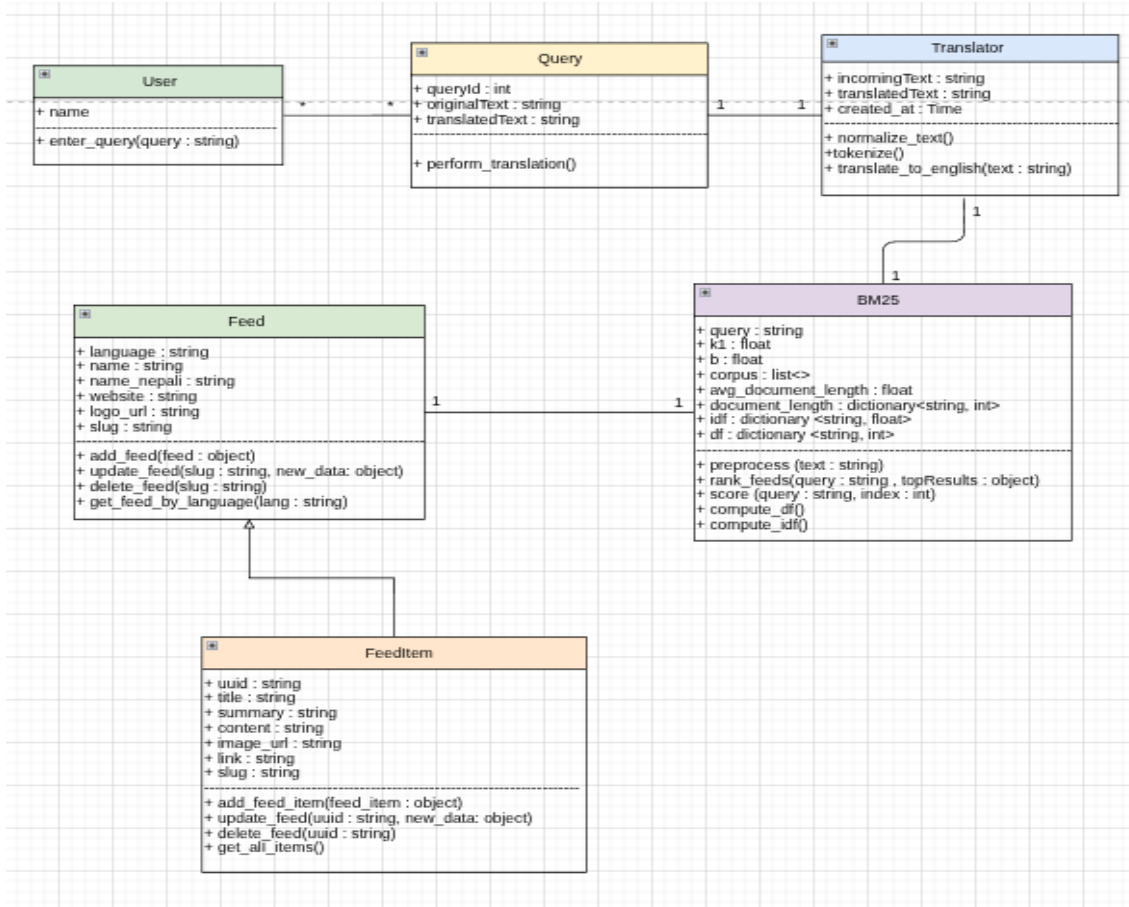


Figure 2.3.1: Class Diagram

3.3.2 State Diagram

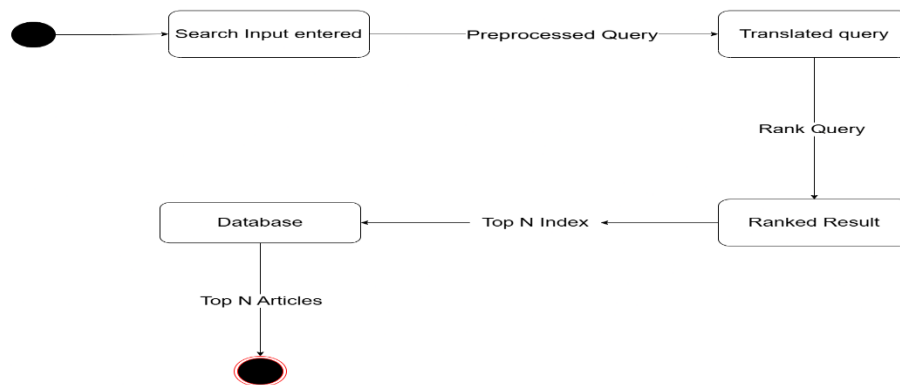


Figure 3.3.2: State Diagram

3.3.3 Sequence Diagram

The sequence diagram shows the workflow in following steps:

1. User enters the query to search.
2. System sends the query to NLTTA model.
3. NLTTA model translates the query and return the translated query to system.
4. System forwards the translated query to BM25.
5. BM25 ranks and return the index of top N documents based on query to system.
6. System requests the top N documents from database using the indexes sent by BM25.
7. The database sends the documents to the system.
8. The top results are displayed to the user.

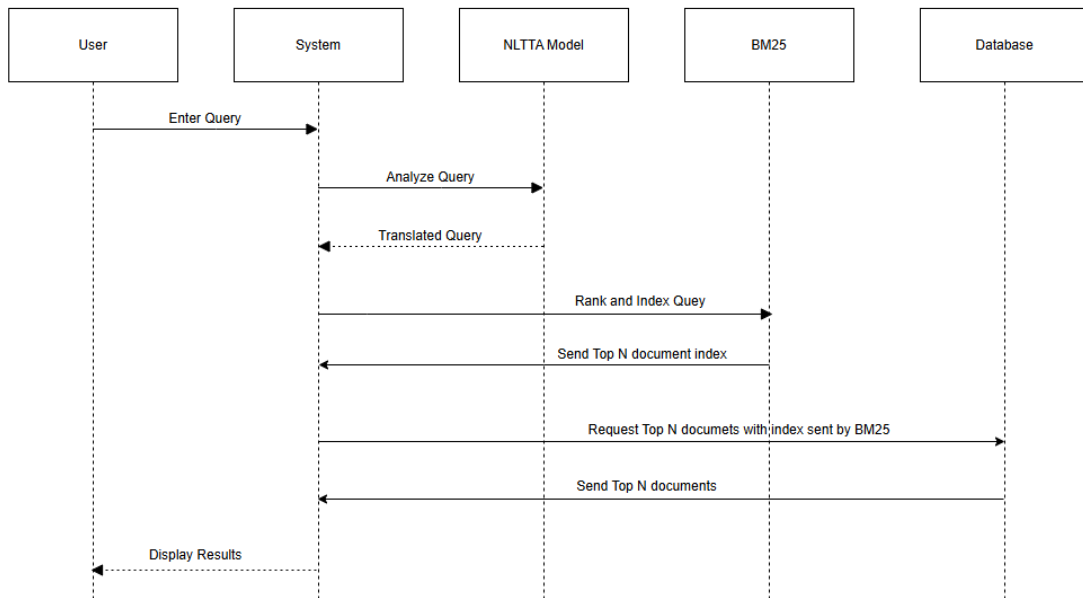


Figure 3.3.3: Sequence Diagram

Chapter 4: System Design

4.1 Design

4.1.1 Component Design

Components:

- ❖ **User Device:** Includes the User Interface (UI) for interaction.
- ❖ **Web Server:** Handles requests through the backend and APIs.
- ❖ **ML Server:** Processes user query using a fine-tuned LLM and retrieval algorithm.
- ❖ **Database Server:** Stores information regarding news feed and articles.

Flow:

- ❖ The user enters the query they want to search.
- ❖ The query is sent to the backend via APIs.
- ❖ The ML Server preprocesses the query and translates it through the NLTTA model.
- ❖ The translated query is used by BM25 retrieval method to get top N indexes.
- ❖ The backend requests the top N indexes documents/articles form database.
- ❖ The documents/articles returned by database are show to users.

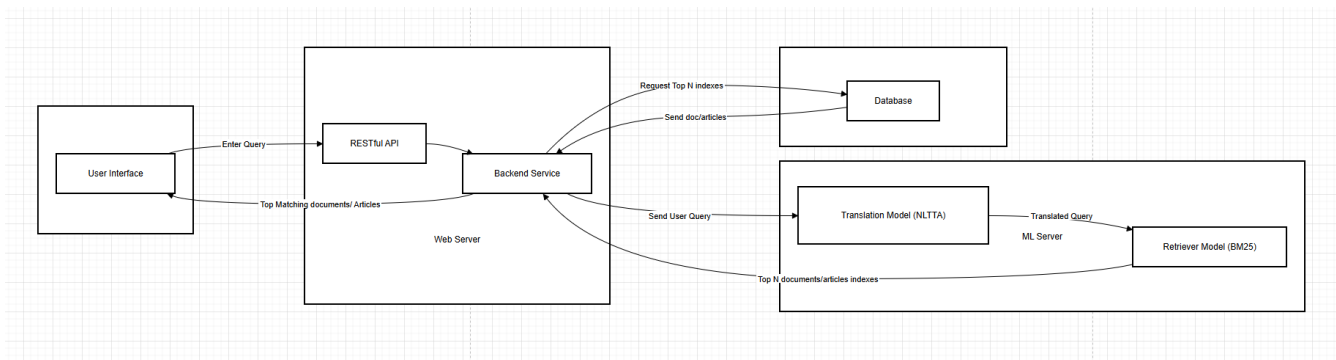


Figure 4.1: Component Diagram

4.2 Algorithm Details

1. Translation Algorithm

- **Approach:** Neural Translation
- **Method:** Fine Tuning on a domain
- **Model:** mBART based on BART architecture
- **Steps:**
 1. Preprocess and prepare the data:
 - Normalization, add special token for translation of source language like >>ne_XX<< to roman sentences and >>en_XX<< for English.
 2. Load pre-trained tokenizer and mBART model from open-source resources, also set the source and target language to our special tokens.
 3. Create Data Collator for training the Seq2Seq model using the preprocess dataset.
 4. Configure Seq2Seq trainer with all suitable parameters like model, learning rate, epochs, output_dir, etc.
 5. Train the model using our data and Seq2Seq trainer.
 6. Evaluate and save.

2. Recommendation Algorithm:

- **Approach:** Probabilistic retrieval
- **Method:** Term frequency saturation and document length normalization
- **Model:** BM25
- **Steps:**
 1. Indexing:

- Compute document frequencies $df(t)$ for each term 't' in the corpus of N documents.
- Compute average document length:

$$avgDL = \frac{1}{N} \sum_D |D|$$

2. Compute IDF for Each Term:

$$IDF(t) = \ln \frac{N - df(t) + 0.5}{df(t) + 0.5}$$

3. Set Hyper-parameters:

- k_1 : controls term-frequency scaling (commonly $1.2 \leq k_1 \leq 2.0$).
- b : controls document-length normalization (commonly $b=0.75$).

4. For Each Query QQQ, For Each Candidate Document DDD:

- For each query term $q \in Q$, get its term frequency in the document $f(q,D)$.
- Compute the term's BM25 contribution:

$$Score_q(D) = IDF(q) * \frac{f(q,D)(k_1+1)}{(f(q,D) + k_1) (1 - b + b * \frac{|D|}{avgDL})}$$

5. Aggregate Term Scores:

$$BM25(D, Q) = \sum_{q \in Q} score_q(D)$$

6. Rank Documents:

- Sort all candidate documents by their BM25 scores in descending order.
- Return the top-kkk highest-scoring documents as the retrieval result.

Chapter 5: Implementation and Testing

5.1. Implementation

5.1.1 Tools Used

- **CASE Tools:** Visual Studio Code, Postman (API Testing).
- **Programming Languages:** Python (backend and model development), JSX.
- **Frameworks:** Django (backend), NextJS (frontend)
- **Database Platform:** MYSQL
- **Machine Learning Tools:** PyTorch, Transformers.

5.1.2 Implementation Details of Modules

1. Query Input

- **Classes/Functions:**
 - Query: Manages searched query.
 - Perform_translation(): Sends image file and metadata.

2. Translation Module

- **Algorithm:**
 - Load NLTTA Model.
 - Preprocess searched Query.
 - Translate tokenized query.
 - Return decoded string.

3. Retrieval Module

- **Algorithm:**
 - Load BM25 model and initialize corpus if not done previously.
 - Compute score for all documents using query

- Rank the top N documents/ articles based on score
- Return top N documents indexes

4. Frontend Module

○ Components:

- UI form for query.
- Search result page for displaying top matching articles/documents.

5.2 Testing

5.2.1 BLEU and BERT Score

BLEU score is a metric used to evaluate the quality of machine-translated text by comparing it to one or more reference translations. It measures the overlap of n-grams between the candidate and reference, with higher scores indicating better translation accuracy.

BERT score evaluates text similarity by comparing contextual embeddings of words using a pre-trained BERT model. It captures semantic meaning and aligns words based on their context rather than exact surface forms.

BERT score measures deep semantic similarity, making it more robust to synonyms and paraphrasing. BLEU relies on exact n-gram matches, which may penalize valid translations with different wording.

Our BLEU score test was conducted using a list of roman Nepali inputs corresponding to their English references.

Table 5.2.1 Test Cases

Test Case ID	Description	Input (Roman Nepali)	Expected Output (English)	Actual Output	Status	Remarks
TC_001	Translate a simple greeting	"Namaskara"	"Greetings"	"Greetings"	Pass	Basic word translation

Test Case ID	Description	Input (Roman Nepali)	Expected Output (English)	Actual Output	Status	Remarks
TC_002	Translate a common phrase	"Timi kasto chau?"	"How are you?"	"How are you?"	Pass	Simple sentence with question
TC_003	Translate with mixed vocabulary	"ma school janchu"	"I go to school"	"I go to school"	Pass	Correct phrase with verb tense
TC_004	Translate unknown/ambiguous words	"Chaina"	"No"	"Not available"	Fail	Check handling of ambiguous words

5.3 Result Analysis

5.3.1 BLEU Score Result:

- On conduction of BLEU score test on randomly selected and generated data, our model got a score of 39.84 which is regarded as good translations.
- Although the other models with 600 times the parameter of our model achieved a certain raise in performance but the scalability and the cost of running those make our model as one of the best options.

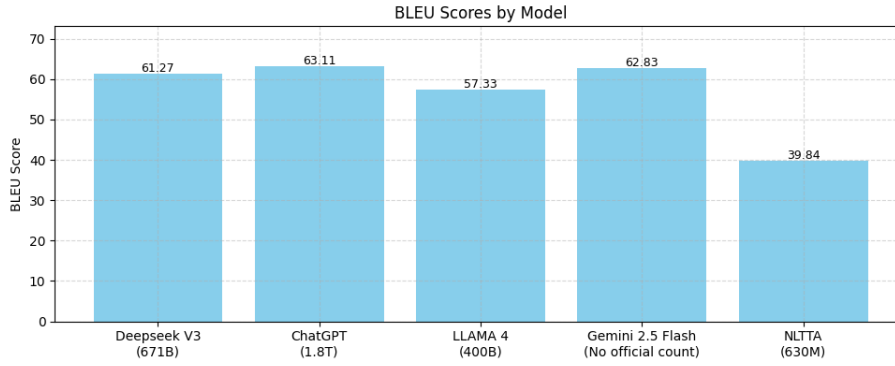


Figure 5.3.1: BLEU score vs Other Models

5.3.2 BERT Score Result:

- After the model evaluation we got a BERT score of 0.9613, which is regarded as excellent.
- The other bigger and paid models got around the mark of 0.97 thus only a slight improvement on our model. Hence making our model the standout even with limited size and resources.

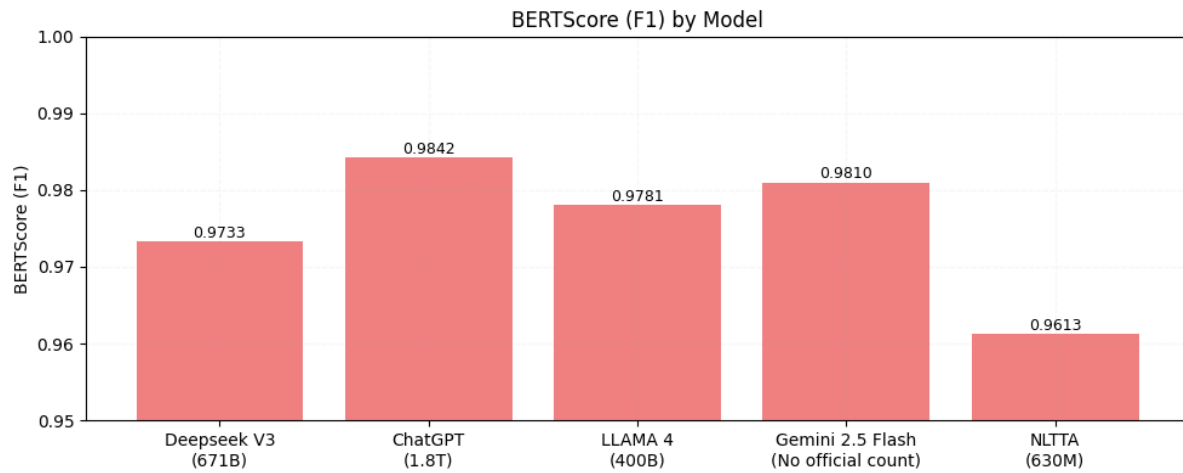


Figure 5.3.2: BERT score vs Other Models

Chapter 6: Conclusion and Future Recommendations

6.1 Conclusion

The NLTTA web application successfully addresses the critical challenge of bridging the linguistic gap for Nepali users in the digital space. By leveraging advanced deep learning techniques, particularly transformer-based architectures like mBART, the system effectively processes Nepali inputs in both Devanagari and Romanized scripts, as well as mixed Nepali-English queries.

The application enables users to search for news content seamlessly, even when using informal, chat-style language. This significantly enhances digital inclusion, allowing individuals with limited English proficiency to access relevant online information easily. NLTTA ensures scalability for integrating additional features in the future. Overall, the project demonstrates promising results in improving digital accessibility and empowering a wider spectrum of Nepali-speaking users to engage meaningfully with online platforms.

6.2 Future Recommendations

- 1. Expansion to Other Domains:**

Integrate NLTTA with additional applications beyond news search, such as e-commerce platforms, government portals, and educational tools, to broaden its utility.

- 2. Enhanced Informal Language Handling:**

Improve the model's capability to understand slang, dialect-specific expressions, and colloquial phrases commonly used in Nepali digital communication.

- 3. Multilingual and Multimodal Support:**

Extend support to other regional languages spoken in Nepal and explore processing speech inputs for voice-based searches and commands.

- 4. Collaboration with Institutions:**

Partner with educational and governmental institutions to promote NLTTA as a tool for enhancing digital literacy and bridging linguistic barriers nationwide.

- 5. Continuous Model Updating:**

Regularly retrain and fine-tune the translation and retrieval models using updated datasets to maintain high accuracy and adapt to evolving language usage trends.

REFERENCES

- [1] D. Bahdanau, K. Cho, and Y. Bengio, “Neural Machine Translation by Jointly Learning to Align and Translate,” Sep. 2014, [Online]. Available: <http://arxiv.org/abs/1409.0473>
- [2] S. Khanuja *et al.*, “MuRIL: Multilingual Representations for Indian Languages,” Mar. 2021, [Online]. Available: <http://arxiv.org/abs/2103.10730>
- [3] Y. Liu *et al.*, “Multilingual Denoising Pre-training for Neural Machine Translation,” Jan. 2020, [Online]. Available: <http://arxiv.org/abs/2001.08210>
- [4] T. Kudo and J. Richardson, “SentencePiece: A simple and language independent subword tokenizer and detokenizer for Neural Text Processing,” Aug. 2018, [Online]. Available: <http://arxiv.org/abs/1808.06226>
- [5] A. Vaswani *et al.*, “Attention Is All You Need,” *31st Conference on Neural Information Processing Systems*, Jun. 2017, [Online]. Available: <http://arxiv.org/abs/1706.03762>
- [6] T. Mikolov, K. Chen, G. Corrado, and J. Dean, “Efficient Estimation of Word Representations in Vector Space,” Jan. 2013, [Online]. Available: <http://arxiv.org/abs/1301.3781>
- [7] S. Pudasaini, S. Shakya, A. Tamang, S. Adhikari, S. Thapa, and S. Lamichhane, “NepaliBERT: Pre-training of Masked Language Model in Nepali Corpus,” in *7th International Conference on I-SMAC (IoT in Social, Mobile, Analytics and Cloud), I-SMAC 2023 - Proceedings*, Institute of Electrical and Electronics Engineers Inc., 2023, pp. 325–330. doi: 10.1109/I-SMAC58438.2023.10290690.

APPENDIX

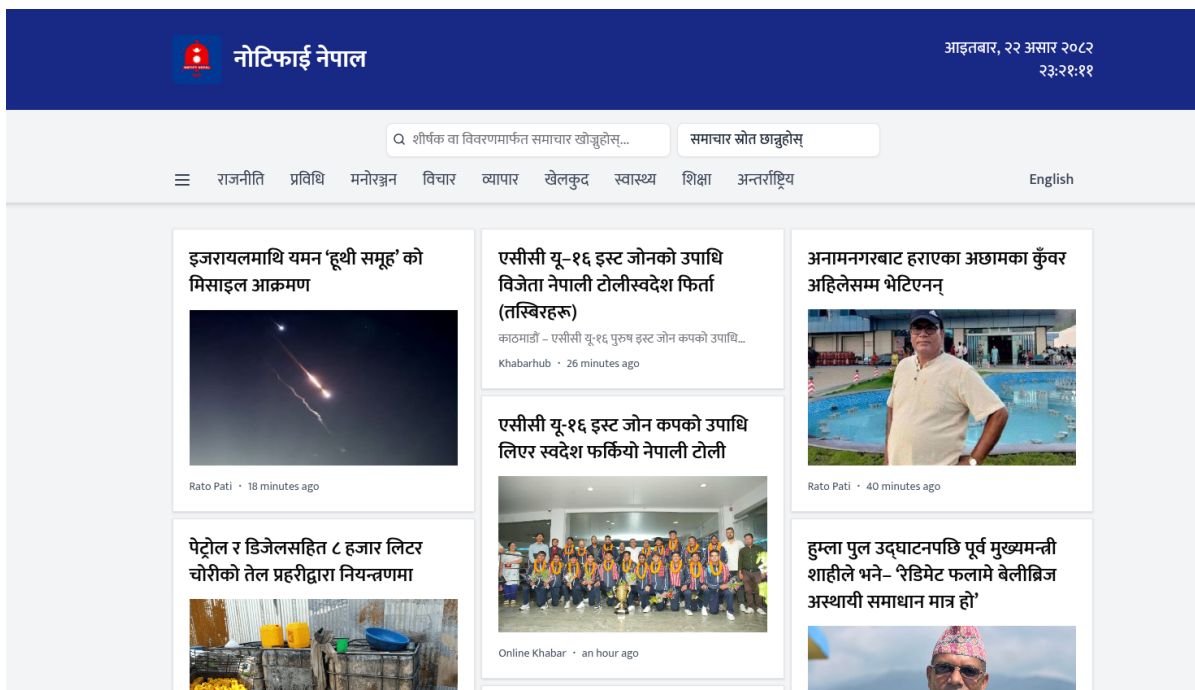


Figure 1:Home Page

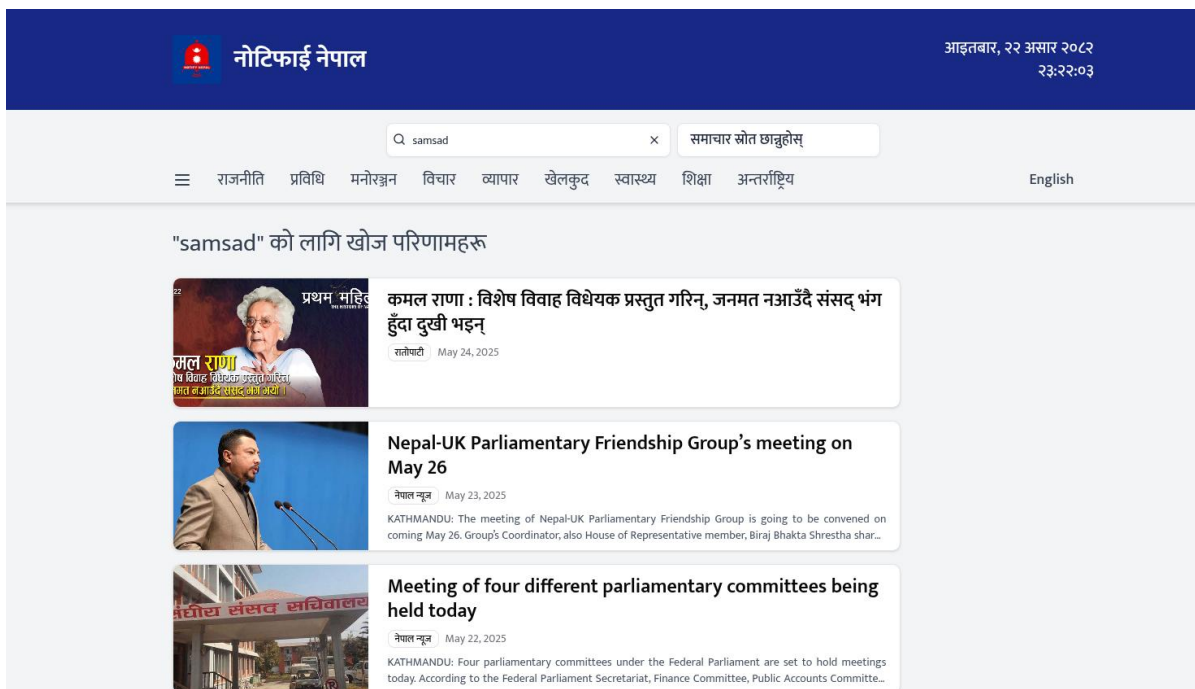


Figure 2:Searching samsad for parliament

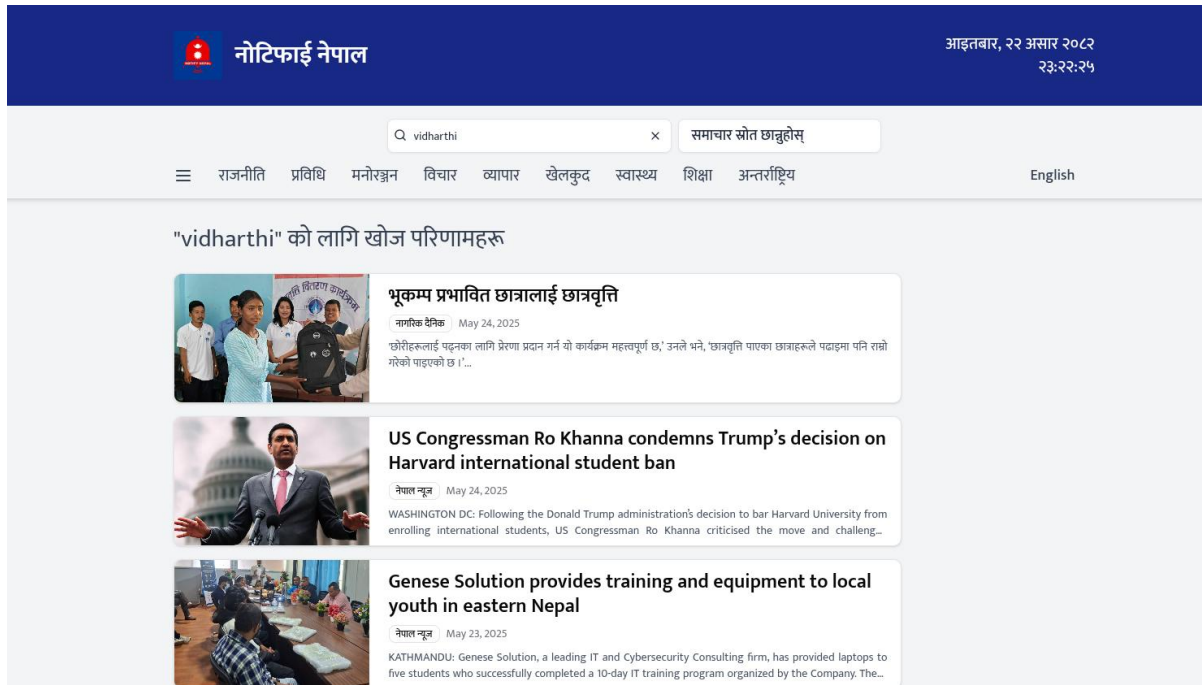


Figure 3: Searching Vidharthi for student



Figure 4: Article Page



Figure 5: Searching Ghar Mantri for HomeMinister



Figure 6: Why Nepali article comes with home minister